

Formal Methods for Software Engineering homework report

Authors: Davide Greco, Edoardo Egidio

Table of Contents

1. Design choices	2
1.1. Master-slave approach	2
1.2. World movement (symbolic)	2
1.3. Player modeling (symbolic)	3
2. Symbolic version results	4
2.1. First query	4
2.1.1. Assignment	4
2.1.2. Query used	4
2.1.3. Results	4
2.2. Second query	4
2.2.1. Assignment	4
2.2.2. Query used	4
2.2.3. Results	4
2.3. Third query	4
2.3.1. Assignment	4
2.3.2. Query used	4
2.3.3. Results	4
2.4. Fourth query	5
2.4.1. Assignment	5
2.4.2. Query used	5
2.4.3. Results	5
2.5. Fifth query	5
2.5.1. Assignment	5
2.5.2. Query used	5
2.5.3. Results	5
2.6. Sixth query	5
2.6.1. Assignment	5
2.6.2. Query used	5
2.6.3. Results	5
2.7. Seventh query	5
2.7.1. Assignment	5
2.7.2. Query used	5
2.7.3. Results	5

1. Design choices

1.1. Master-slave approach

Since at every tick both the world and the player must move, we decided to take a master-slave approach: at each tick, the world moves first, and notifies a global urgent channel, from which the player listens. As soon as the world ticked, the player performs its “turn”, which is:

- checking if the world movement killed him
- if not, perform a movement (which can either be to go up or to stay still)
- check if the movement killed him:
 - if so, we must reset
 - if not, we go back to waiting state (unless we are in `MAX_Y` row, in which case we just occupied a home bay and we must reset)

This way we are sure that the world and player movement are synchronized, and that there is never a configuration that would kill the player that goes unnoticed.

1.2. World movement (symbolic)

Since all elements of a row move at the same speed (at least for the symbolic version, see [link to stochastic handling of trucks](#)), to reduce state-space size we decided to model the world as an array of rows that move with the speed of the elements occupying the given row. To achieve this we used:

- A matrix containing the initial state of the world: each row has $18 + 2$ columns, to handle the warping that takes two more steps. Each cell is an int representing the type of the cell, which can be

Number	Name	Description
-1	<code>C_WARP</code>	not a real cell, but the space of warping
0	<code>C_EMPTY</code>	a cell in which there is no entity (not considering the player)
1	<code>C_OBJ</code>	either a car (in the first 6 rows), or a wall (in the last row)
2	<code>C_LOG</code>	a log (only in rows 7-11)
3	<code>C_TURTLE</code>	a regular (i.e. not diving) turtle (only in rows 7-11)
4	<code>C_DIVING</code>	a diving turtle (only in rows 7-11)

- An array of speeds (one for each row). As per specification, the speed of a row is the number of ticks between two consecutive movements.
- An array of directions (one for each row). When we want to get a specific cell, we need to multiply the current offset with the direction of the current row. We must however pay attention to a detail: without loss of generality, let's assume that we are looking at a row that moves from left to right. Since we are moving the whole row, when it moves all its elements will be shifted to the right by one, so the element now occupying cell `x` is the element in the original matrix occupying cell `x-1`: this means that we must use `-1` for left-to-right movement, and `1` for right-to-left movement.
- An array of offsets (one for each row). Each tick, if the current number of ticks is divisible by the speed of the current row, this will increase by one (with modulo `MATRIX_COLS`).
- The function `int[-1,4] get_cell(int[0, MAX_Y] row, int[0, MATRIX_COLS] col)`, which makes use of all the aforementioned structures to get the type of cell currently occupying a given position.

In this way, we have a single automata for all the rows that, at each ticks, updates all the rows (and hence all the elements of the rows). Moreover, it also results in a very simple automata: every tick of its clock, update the world according to the rules we just described and notify the channel. One other thing this automata checks is the state of the diving turtles: since we only care about what the turtles do to the player, we only care about the moments in which they kill the player (submerged), and the moments they don't kill the player (submerging, emerging, emerged). This is handled by a boolean, `are_diving_turtles_up`, which gets set according to the time units described in the specification.

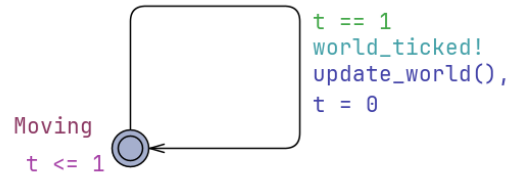
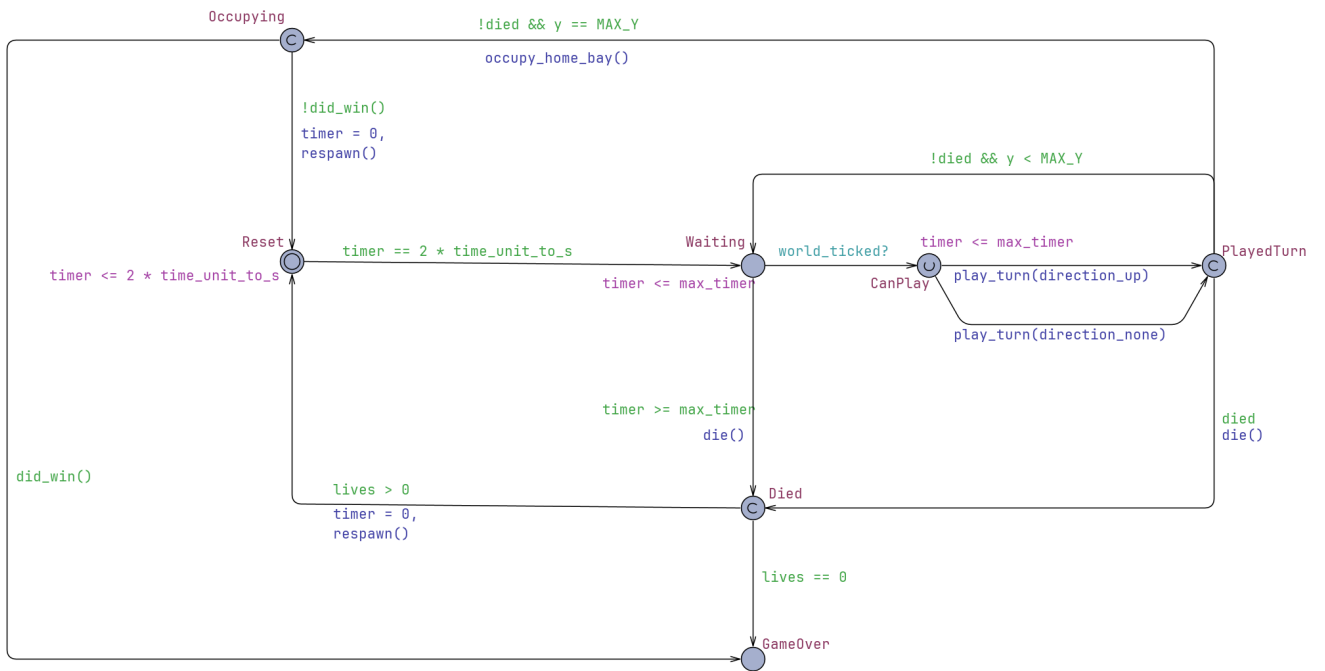


Figure 1: World automata

1.3. Player modeling (symbolic)

The player needs to handle a lot of different scenarios, so it will naturally be more complex:



We start in the `Reset` state, and we stay there for 2 seconds (which corresponds to 20 time units). After that time period, it goes in the `Waiting` state, from which it can exit only if one of two conditions is met:

- the `world_ticked` channel pushed a notification: we must perform the player turn
- the timer reached the maximum value allowed: it counts as player death

In the former, the turn is the one described in [Section 1.1](#). If the player reached the last row, it means it occupied a home bay: this means we must mark the home bay as occupied (hence making it unreachable in the next round), and check if we won the whole level. If that is the case, in the symbolic mode we must stop. The same applies if the player dies and its lives reach 0.

2. Symbolic version results

Note that there are two different system declarations: the “complete” one, and the “partial” one. For the former, ensure that in `System declarations` the line `system frogger, world;` is uncommented and `system world;` is commented; while for the latter the opposite must be verified. The first two queries will use “partial” system, while the others the “complete” one.

2.1. First query

2.1.1. Assignment

the game never reaches a deadlock state

2.1.2. Query used

```
A[] not deadlock
```

2.1.3. Results

The query is trivially verified.

2.2. Second query

2.2.1. Assignment

all vehicles and floating objects can only move inside their predefined row

2.2.2. Query used

```
A[] (offsets[1] >= 0 && offsets[1] < MATRIX_COLS &&
      offsets[2] >= 0 && offsets[2] < MATRIX_COLS &&
      offsets[3] >= 0 && offsets[3] < MATRIX_COLS &&
      offsets[4] >= 0 && offsets[4] < MATRIX_COLS &&
      offsets[5] >= 0 && offsets[5] < MATRIX_COLS &&
      offsets[7] >= 0 && offsets[7] < MATRIX_COLS &&
      offsets[8] >= 0 && offsets[8] < MATRIX_COLS &&
      offsets[9] >= 0 && offsets[9] < MATRIX_COLS &&
      offsets[10] >= 0 && offsets[10] < MATRIX_COLS &&
      offsets[11] >= 0 && offsets[11] < MATRIX_COLS)
```

2.2.3. Results

The query is true by construction since the rows themselves move, but we can easily run it and see it is verified.

2.3. Third query

2.3.1. Assignment

Frogger can reach the home bay in position (0,12)–(1,12)

2.3.2. Query used

```
E<> frogger.y == 12 && (frogger.x == 0 || frogger.x == 1)
```

2.3.3. Results

Easily verified: 0.324s

2.4. Fourth query

2.4.1. Assignment

Frogger can score 17 points reaching the home bay in position (8,12)–(9,12)

2.4.2. Query used

```
E<> frogger.points == 17 && frogger.home_bays_occupied[2] &&  
      frogger.y == 12 && (frogger.x == 8 || frogger.x == 9)
```

2.4.3. Results

We need to check the point immediately after occupying the home bay, so we must check that the home bay is marked as occupied (so its points have been added), and the position is still the one required (no other movement). Verified in 0.245 s

2.5. Fifth query

2.5.1. Assignment

Frogger can reach a yellow cell in row 6 after the player has lost two lives

2.5.2. Query used

```
E<> frogger.lives == 3 && frogger.y == 6
```

2.5.3. Results

Easily verified: 0.03s

2.6. Sixth query

2.6.1. Assignment

Frogger may stand on a log (with five lives) 10 seconds after the game starts

2.6.2. Query used

```
E<> frogger.lives == 5 &&  
      frogger.on_cell == C_LOG &&  
      global_clock == 10*time_unit_to_s
```

2.6.3. Results

Verified: 3.756s

2.7. Seventh query

2.7.1. Assignment

Frogger can die riding a diving turtle that transitions to the submerged state

2.7.2. Query used

```
E<> frogger.died_by_submerging
```

2.7.3. Results

This query is simple thanks to the `died_by_submerging` attribute: it is always false, except when we die while on a diving turtle before moving: this means the player died specifically because of the submerging of the turtles. Because of this, it is easily verified: 0.325s